

2.1 Analyze algorithms for correctness

الگورتھم تب درست (Correct) سمجھا جاتا ہے جب دیے گئے ان پٹ پر اس کا آؤٹ پٹ وہی ہو جو ہونا چاہیے۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 01 of 16

2.1 Analyze algorithms for correctness

LEARN IT

An algorithm is **correct** if its **output matches the expected results** for a given input. Analysing correctness means showing that the algorithm produces the intended output for **all valid inputs**, not just one lucky example.

Two methods are used in this course:

Trace tables → paper execution, line by line → Stepwise reasoning → logic blocks, overall state

You already met the idea of a dry run in Class XI. Here you learn to do it formally, and to choose the method that fits the size of the algorithm.

WORKED EXAMPLE

Worked example: what "correct" means. An algorithm is supposed to count how many of the heights 1.91, 2.01, 1.87, 1.57, 1.48 lie between 1.57 m and 1.91 m inclusive.

Expected output: **3** (1.91, 1.87 and 1.57 qualify; 2.01 is too tall; 1.48 is too short).

If a program prints 4, it **runs** but it is **not correct**. Correctness is about the right answer, not about "no crash".

REMEMBER IT

Remember: **runs ≠ correct**. A logical error still produces output — just the wrong output. That is why we analyse correctness on paper before we trust the code.

COMMON MISTAKE

Mistake: testing only one "normal" input and calling the algorithm correct.

Fix: also try edge cases (empty list, $N = 0$, the exact boundary values 1.57 and 1.91). The definition says **all valid inputs**.

اردو میں سمجھیں

الگورتھم تب درست (Correct) سمجھا جاتا ہے جب دیے گئے ان پٹ پر اس کا آؤٹ پٹ وہی ہو جو ہونا چاہیے۔ صرف چل جانا کافی نہیں؛ منطقی غلطی سے غلط جواب بھی نکل سکتا ہے۔ درستگی جانچنے کے دو طریقے ہیں: ٹریس ٹیبل (Trace Table) اور مرحلہ وار استدلال (Stepwise Reasoning)۔

CHECK YOURSELF

1. When is an algorithm considered correct?
2. Name the two methods used to analyse correctness in this chapter.
3. Why is "the program ran without an error" not enough?

Answers: 1. When its output matches the expected results for a given input (and does so for all valid inputs). 2. Trace tables and stepwise reasoning. 3. Because a logical error can still produce output — the wrong output.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An algorithm is correct if:

- a. Outputs match the spec for the intended inputs (incl. edges)
- b. It is short
- c. It uses XOR
- d. It prints Hello

MCQ 2. A failed edge case means:

- a. Not fully correct
- b. Still correct enough always
- c. Faster Big O
- d. Better UX

MCQ 3. You demonstrate correctness in this chapter with:

- a. Trace tables / stepwise reasoning / tests
- b. Only careers
- c. Only Figma
- d. Only pie charts

MCQ 4. A spec is:

- a. The expected behaviour
- b. The CPU model
- c. The font
- d. The sprint name

MCQ 5. Partial correctness often means:

- a. IF it halts, the answer is right — you still need termination
- b. It never needs to halt
- c. It is Agile
- d. It is a GUI

MCQ 6. Off-by-one in a loop is a:

- a. Correctness bug
- b. OSI issue
- c. NAND issue
- d. Career issue

Extra short questions

1. Define correctness.
2. Two ways it can fail.
3. Why test edges.
4. Link to later trace tables.

Extra long question

1. Give a “find maximum” algorithm, two traces (normal and $n=1$), and an incorrect version that fails on negatives. Explain.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. For every specified input, the algorithm produces the specified output and finishes.
2. Wrong answer; infinite loop (does not finish).
3. Empty, one element, already sorted, not found, max int.
4. A dry run is a manual proof for a small case.

Long

1. Init max to first element not 0. Trace [3,-1,5] and [-4]. Bug: init 0 fails all-negative lists. That is correctness, not Big O.